

From: AI Horizon Forecast aihorizonforecast@substack.com
Subject: Toto: A Foundation Time-Series Model Optimized for
Observability Data
Date: 10 Jul 2025 at 22:01:48
To: egonschiele0112@gmail.com

Forwarded this email? [Subscribe here](#) for more

Toto: A Foundation Time-Series Model Optimized for Observability Data

Datadog open-sourced their pretrained time-series model

NIKOS KAFRITSAS

JUL 10



READ IN APP ↗



Foundation time-series models keep pushing the frontier.

Recently, Datadog has just open-sourced **Toto[2]— Time Series Optimized Transformer for Observability**, a model purpose-built for observability data.

Toto is currently the most extensively pretrained time-series foundation model: The pretraining corpus contains 2.36 trillion tokens, with **~70%** coming from Datadog's private telemetry dataset.

If you read my newsletter, you know my main concern with pretrained time-series models: finding large, high-quality data. This is tough, since most

good datasets aren't public.

This article dives into Toto—how it works, and what sets it apart from other time-series foundation models. In Part 2, we'll walk through a tutorial.

Let's get started!

✓ Find my **Toto** use case project in the [AI Projects folder](#) (Project 20).

What is Toto?

Toto is a decoder-only Transformer foundation model, specialized in zero-shot time series forecasting for observability.

Key characteristics:

- **High-quality data:** Toto is a 151M parameter model, pretrained on 2.36 trillion tokens.
- **Probabilistic forecasting:** Uses a Student-t mixture head to capture the heavy tails in observability time-series data.
- **Proportional factorized space-time attention:** An attention mechanism for multivariate series, modeling both time and feature dependencies.
- **Patch-based causal attention:** A modified version of the popular reversible-instance normalization layer, adapted for decoder architectures.

This paper contributes:

1. The open-weight Toto model, now on HuggingFace.
2. The **BOOM dataset** (Benchmark of Observability Metrics), a new and unique dataset with 350M observations across 2807 distinct multivariate time series—twice the size of the GIFT-eval benchmark.

Next, let's explore what makes Toto unique.

Toto's Pretraining Dataset

Finding a high-quality dataset is one of the most critical steps when building a pre-trained model - this is true for every domain, including time-series forecasting.

Toto's edge is the large, unique dataset from Datadog's platform. Compared to the pretraining corpora of the other foundation models, Toto's pretraining corpus is huge:

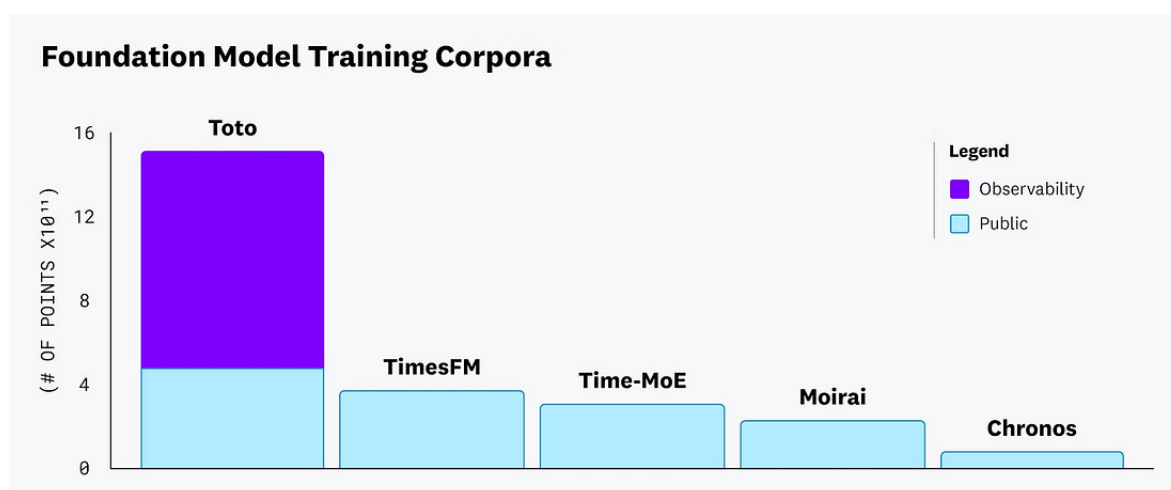


Figure 1: Comparing pretraining corpora of different foundation time-series models. Toto wins by a large margin
(Source [1])

But why is Datadog's observability dataset high-quality?

Datadog collects a wide range of metrics from modern infrastructure and applications - memory usage, CPU load, disk I/O, network throughput, as well as application data like hit counts, error rates, and latency.

It also integrates data from SaaS products, cloud services, open-source frameworks, and third-party tools. This data arrives at high time resolution, often every few seconds or minutes, making the dataset both massive and continuously updated.

Toto's dataset presents several unique characteristics:

- **High time resolution:** Data is collected every few seconds or minutes, unlike many public datasets at hourly or daily frequency.
- **Sparsity and skew:** Many metrics are sparse or zero-inflated, especially error counts, while latency data is highly right-skewed with extreme outliers.
- **Dynamic, non-stationary systems:** The monitored systems change frequently due to deployments, scaling, configuration updates, and user trends.
- **Historical anomalies:** Past incidents and regressions create outliers in telemetry, making such observations hard to find in public datasets.

Toto benefits from the variety and scale of this pretraining dataset - helping the model capture the complex characteristics of real-world observability metrics.

Why use a foundation model for telemetry data?

Time series data with characteristics like sparsity and intermittency isn't

new.

Specialized models—Croston’s method, IMAPA—handle these well. Other models can also be adapted, such as Deep Learning or Tree-based approaches using Poisson or Gamma loss functions.

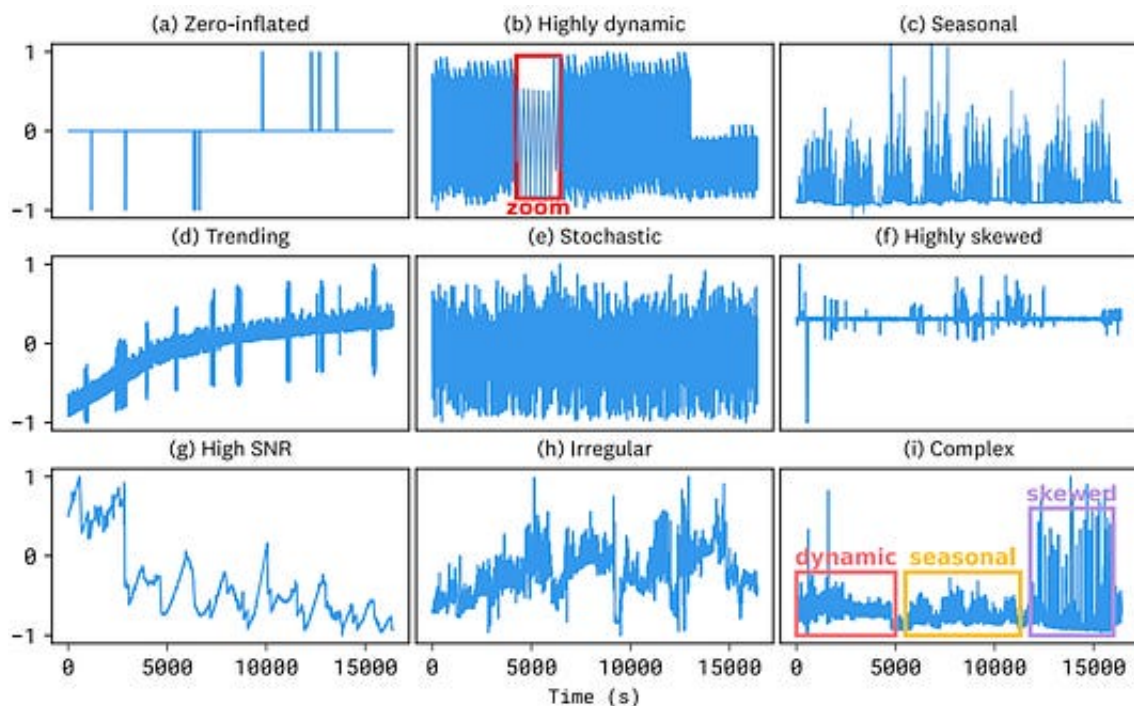


Figure 2: Examples from BOOM showcase the unique time-based patterns present in observability data. (Source [2])

Beyond requiring no training (or minimal finetuning) and matching (or sometimes outperforming) competitive models, there’s another reason:

Short-lived infrastructure components (e.g., an ad-hoc Docker container) often have very limited historical data, making it hard to train effective models. In these cases, time series tend to be very short, so we usually rely on statistical models to fill the gap.

However, conventional alerting systems typically require several days or

weeks to adjust before they can reliably monitor new metrics. Since containers and similar resources exist only for brief periods, these traditional methods cannot adapt quickly enough to be effective. As a result, many real-world systems fall back on simple rule-based alerts that depend heavily on user expertise.

Zero-shot foundation models address this challenge by leveraging patterns learned from a large and diverse dataset, enabling accurate predictions even with minimal historical data.

Proportional factorized space-time attention

Most foundation time-series models usually attempt to improve the attention mechanism - which is the heart of the Transformer.

Toto also addresses this by introducing its own attention mechanism.

[In the previous article](#), I made a brief recap of how attention evolved in Transformer-based models. The current state-of-the-art is to apply attention across both the time and feature dimension - by considering multiple variables and also making this operation less costly.

The authors noticed that both approaches are beneficial for enhancing accuracy - however models that are univariate and ignore feature dependencies are still competitive on multivariate datasets.

Therefore, since time-wise dependencies are more important than space-wise (dependencies among features), Toto uses a mixture of alternating space-wise and time-wise attention blocks - with one *space-wise attention block*(purple) for every 11 *time-wise blocks* (blue) **(Figure 3)**:

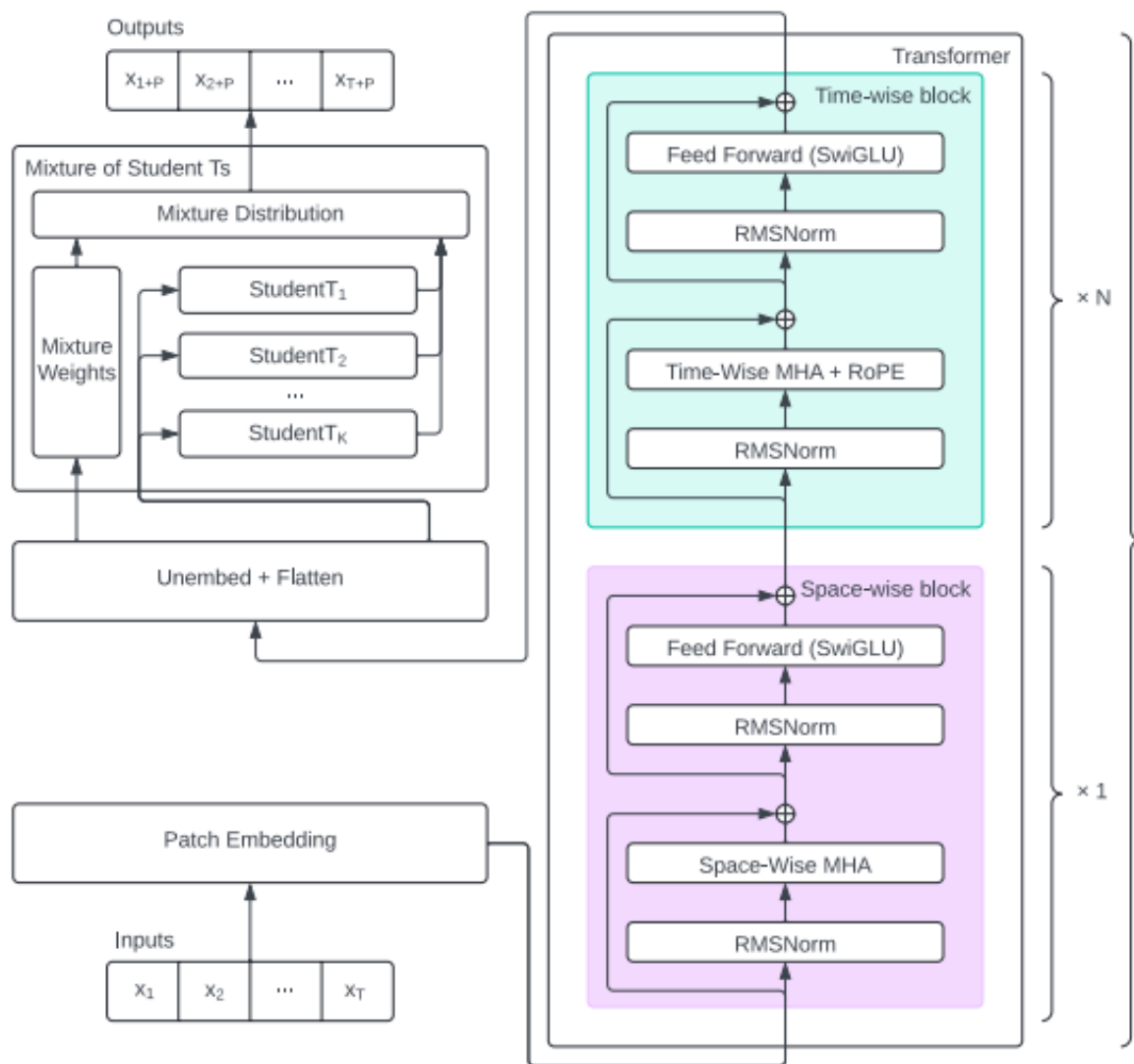


Figure 3: The Transformer blocks (Time-wise and Space-wise) in Toto's architecture (Source [2])

The ratio of time-wise and space-wise is then configurable - by default, this ratio is 11:1. Both are transformer blocks, but with a few differences:

- **Time-wise blocks** apply causal attention, using autoregressive, decoder-only processing. They also use ROPE embeddings.
- **Space-wise blocks** use bidirectional attention (looking for context in both ways). Since they model relationships between features

(covariates), they maintain permutation invariance—changing the order of features should not affect the forecasting results..

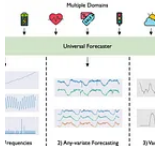
A precursor to this approach is MOIRAI's **Any-Variate Attention** and Timer-XL's **TimeAttention**. I've analyzed both in detail here:



Timer-XL: Long-Context Foundation Model for Time-Series Forecasting

NIKOS KAFRITSAS · JUN 5

[Read full story](#)



MOIRAI: Salesforce's Foundation Transformer For Time-Series Forecasting

NIKOS KAFRITSAS · MARCH 15, 2024

[Read full story](#)

Note: *The authors' observation—that univariate models ignoring feature dependencies still perform well on multivariate datasets—is valid, but mostly on simple datasets (as shown in the TSMixer paper when comparing TSMixer vs TSMixer-Ext, which we discussed [here](#)). On more complex datasets (e.g. retail forecasting), and given that the space-wise/time-wise ratio is configurable, it may be worth increasing the number of space-wise blocks.*

Toto Architecture

Toto is a modern Transformer-based model, using all the latest advancements.

For example, Toto leverages **SwiGLU** activation to capture richer feature interactions with minimal extra compute and **RMSNorm** to stabilize and speed up training by normalizing activation scales without costly mean

subtraction - both SwiGLU and RMSNorm are used by Llama.

Also, Toto processes the input in **patches** - a patch is essentially a token, a sequence of datapoints. Many TF-based models we've covered in this newsletter use patches.

The complete Toto architecture is shown in **Figure 4**:

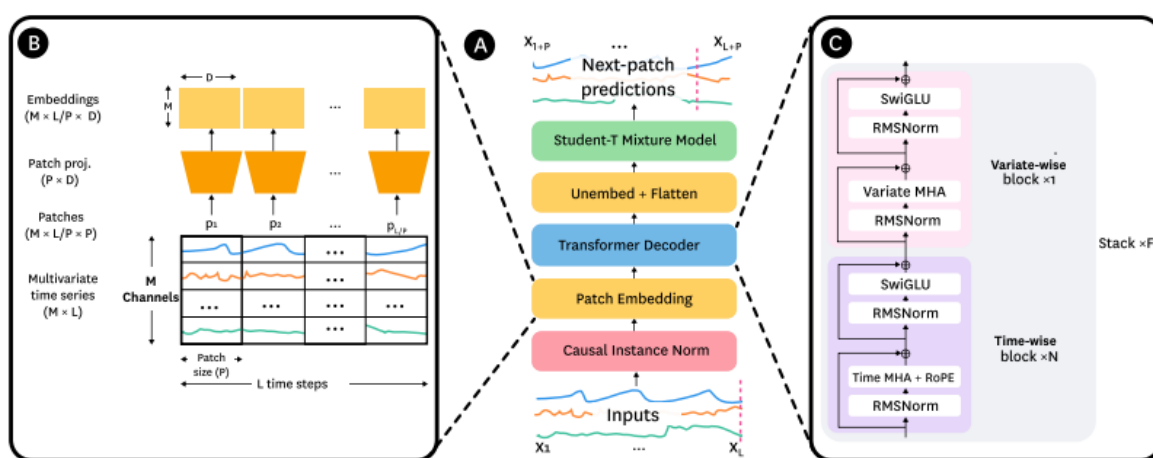


Figure 4: Top-level view of Toto (Source [3])

Below are the main operations in Toto:

- **Input processing:** An L -step, M -variate series is first scaled with causal patch-wise instance normalization, converted into patch embeddings, and fed to a decoder-only Transformer.
- **Patch embedding:** Slices the time axis into $P=64$ -step patches and linearly projects each to a $D=768$ space, yielding a tensor $(M \times L/P \times D)$.
- **Transformer core:** Uses proportional factorized attention: $F=1$ segment comprising $N=11$ time-wise blocks followed by one variate-wise block.
- **Output head:** A Student-T mixture, trained with a composite robust

loss, produces probabilistic next-patch predictions.

Learning a Student-T mixture to regress the output is very smart - and it's something that MOIRAI also does, except MOIRAI uses other distributions as well (a weighted combination of Gaussian, Student-T, Log-Normal, and Negative-Binomial outputs).

Essentially, Toto predicts the parameters of k Student-T distributions (where k is an hyperparameter) for each time step, as well as a learned weighting. The Student-T distribution is more suitable for heavy-tailed time series, which benefits observability and telemetry data.

During inference, the model draws samples from the mixture distribution - meaning we can also predict intervals in the form of quantiles. Since Toto is autoregressive, the outputs of time-step t are fed back to the decoder for the next predictions at step $t+1$.

Remember, Toto processes inputs as patches and, during training, learns to predict the distribution of values in the next patch given all previous patches

Note: Toto predicts 3 parameters for each K Student T distribution - the ν degrees of freedom, mean μ , and scale τ - as well as the weighting parameters π , which sum to 1

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{T}(x \mid \mu_k, \tau_k, \nu_k)$$

By expanding T we have:

$$\mathcal{T}(x \mid \mu, \tau, \nu) = \frac{\Gamma\left(\frac{\nu+d}{2}\right)}{\Gamma\left(\frac{\nu}{2}\right) (\nu\pi)^{d/2} |\tau|^{1/2}} \left(1 + \frac{1}{\nu} (x - \mu)^\top \tau^{-1} (x - \mu)\right)^{-\frac{\nu+d}{2}}$$

where Γ is the gamma function. Toto is trained by minimizing the negative log likelihood (NLL) of the composite loss.

Toto 2025 vs Toto 2024 variants

There's a small detail that's easy to miss.

Datadog released the whitepaper of Toto in July 2024[2], without releasing the model. Last month, Datadog released the model binaries, including a new paper[3].

Interestingly, we notice a few differences between the 2 versions. **Table 1** shows Toto's 2025 hyperparameters, while **Table 2** lists the settings used in the original July 2024 release:

Hyperparameter	Value
Embedding Dimension	768
MLP Dimension	3072
# Layers	12
# Heads	12
# Variates	32
Spacewise Layer Cadence	12
Patch Size	64
# $\mathcal{T}$ Mixture Model Components	24
Annealing Schedule	WSD
Optimizer	AdamW
(β_1, β_2)	(0.9579, 0.9581)
Weight Decay	0.0014
Initial Learning Rate	0.0005
Warmup Steps	6784
Stable Steps	112,255
Decay Steps	15,962
Batch Size	128
Total Train Steps	135,001
$\mathcal{L}_{\text{Robust}} \alpha$	0.0000
$\mathcal{L}_{\text{Robust}} \delta$	0.1010
λ_{NLL}	0.5755
κ	10

Table 1: Toto’s list of hyperparameters (2025 version)

Hyperparameter	Value
Embedding Dimension	512
MLP Dimension	2048
# Layers	24
# Heads	8
# Variates	32
(β_1, β_2)	(0.9, 0.95)
Weight Decay	0.01
Spacewise Layer Cadence	3
Patch Size	32
# Student-T Mixture Model Components	16
Initial Learning Rate	0.001
Annealing Schedule	Cosine
Batch Size	192
Warmup Steps	5000
Total Train Steps	193000

Table 2: Toto's list of hyperparameters (2024 version)

For example, notice that the patch size increased (32→64). Most other foundation models have a smaller patch size. From my experience, a larger patch size makes the model more capable in long-term forecasting, but decreases its efficiency in higher frequencies.

Also, space-wise layers reduced: 1 per 2 → 1 per 11 time-wise layers.

Additionally, the authors stabilize the training process by combining the minimization of the composite loss we showed previously (actually the negative log likelihood NLL) with a Cauchy loss function.

In any case, these changes probably made Toto better, but we don't have last year's variant available to compare their performances.

Ablation Studies and Scaling

A critical difference between Toto-2025 vs Toto-2024 is normalization.

Toto-2024 applied **Reversible Instance Normalization** (RevIN), a popular type of normalization in DL forecasting models. We explained how RevIN works and how it's compared with other types of normalization [here](#).

Toto dropped RevIN in favor of a new method: **patch-based causal instance normalization**.

The issue with global instance normalization in decoder-only architectures is information leakage — normalizing the entire series includes future time steps, breaking causality during next-patch prediction, and degrading performance. As shown in my previous article, [Timer-XL](#) suffered under RevIN for this very reason.

One workaround is to normalize using only the first patch's statistics (used by TimesFM) — this maintains causality but fails on highly nonstationary data, like observability series, where data distributions shift over time.

Toto's solution: apply instance normalization **per patch**, using only the current and past data. For each timestep t , we compute the **causal mean** ($\hat{\mu}_t$) and **causal variance** ($\hat{s}_t$) from the input values x_i and corresponding weights w_i . Weights are set to 0 for padding, 1 elsewhere.

$$\hat{\mu}_t = \frac{\sum_{i=1}^t w_i x_i}{\sum_{i=1}^t w_i}, \quad \hat{s}_t = \sqrt{\frac{\sum_{i=1}^t w_i (x_i - \hat{\mu}_t)^2}{\sum_{i=1}^t w_i - 1} + 0.1}$$

To avoid over-scaling and potential numerical instability, a minimum value of 0.1 is added to the standard deviation. This method preserves causality — and adapts better than static, per-variable scaling.

Of course, the authors perform ablation studies to evaluate their

architectural choices. Not using *patch-based causal instance normalization* has the most negative impact. The full ablation results are shown in **Figure 5**:

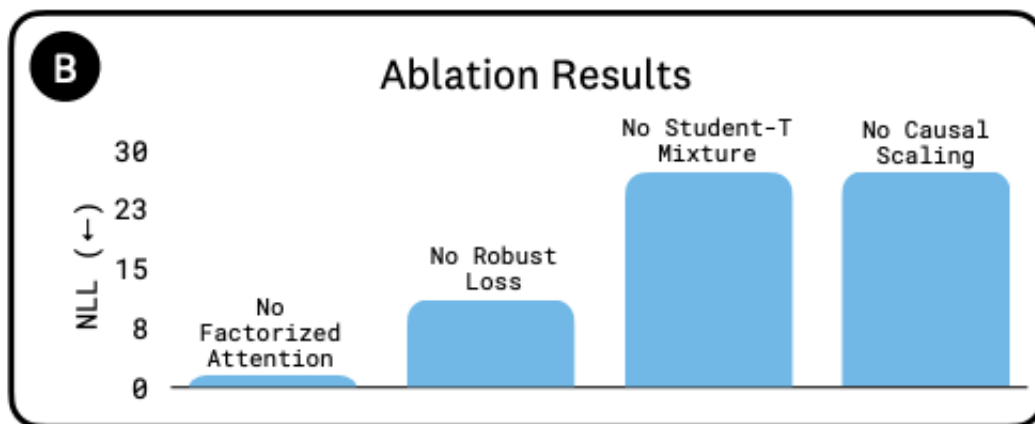


Figure 5: Ablation results showing the impact of each architectural choice on negative log-likelihood (NLL) — higher values indicate worse performance. (Source [3])

Also, replacing the Student-T mixture with a single Student-T output increases NLL by 27.2%. Removing the robust loss component and optimizing NLL alone results in an 11.1% increase, likely due to the stabilizing effect of the robust loss.

Making all attention layers time-wise — effectively removing variate-wise attention while keeping parameter count constant — leads to only a smaller NLL increase of 1.6%. That’s why the authors emphasize using time-wise layers instead of space-time layers

Toto’s Pretraining and Evaluation Dataset

Toto’s pretraining dataset is remarkable, containing:

- **GIFT-Eval Pretrain dataset** (derived *in part* from LOTSA and other

public datasets).

- **Chronos collection**—excluding any datasets that overlap with the GIFT-Eval benchmark to prevent test-data leakage.
- Synthetic Data.

To prepare time series data for training, padding and masking are applied to align series lengths with the patch stride. Data augmentation techniques like random time offsets and variate shuffling are used to increase robustness and prevent memorization. Finally, variates from different datasets—including observability, open-source, and synthetic—are mixed with a 14% probability to enhance diversity and generalization.

Regarding evaluation, the authors used:

- The BOOM dataset I mentioned earlier.
- A random sample of BOOM, called BOOMLET.
- The GIFT-eval (not the pretrain split of course).
- The Long Sequence Forecasting (LSF), introduced in the Informer paper.

Figure 6 shows some key characteristics of the datasets above:

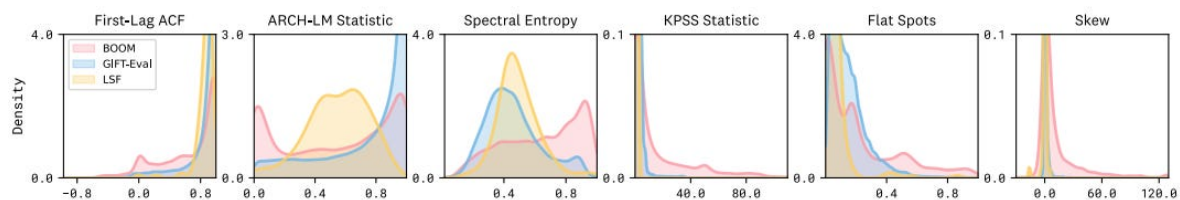


Figure 6: *BOOM and GIFT-Eval datasets show wider and shifted feature distributions than LSF, indicating greater diversity, irregularity, and nonstationarity. (Source [3])*

Since BOOM contains observability data, it's trickier than the others. For example, BOOM's spectral entropy peaks near 1, indicating more high-entropy (noisy, unstructured) signals – typical of observability data.

Benchmark Results

The following benchmarks contain the results on the aforementioned datasets.

The paper included a vast array of time-series models, including the newer versions of the recent foundation models, such as TimesFM-2.0 (which many papers avoid).

Also, for the first 2 benchmarks, the authors evaluate both point forecasting (MASE) and probabilistic forecasting (CRPS). Let's view the results:

Dataset	Metric	Zero Shot							Baselines		
		TOTO	Moirai _{Base}	TimesFM _{2.0}	Chronos _{Bolt-Base}	Time-MoE _{Large}	Timer	VisionTS	Auto-ARIMA	Auto-ETS	Auto-Theta
BOOM	MASE ↓	0.617	<u>0.710</u>	0.725	0.726	0.881	0.796	0.988	0.824	0.842	1.123
	CRPS ↓	0.375	<u>0.428</u>	0.447	0.451	0.643	0.639	0.673	0.736	1.975	1.018
	Rank ↓	2.369	<u>4.328</u>	5.243	5.576	9.843	9.920	10.989	9.686	11.671	12.472
BOOMLET	MASE ↓	0.617	0.779	<u>0.685</u>	0.711	0.793	0.807	0.912	0.922	0.969	1.030
	CRPS ↓	0.519	0.630	<u>0.603</u>	0.637	0.780	0.793	0.885	0.880	15.664	1.182
	Rank ↓	1.244	4.644	<u>4.178</u>	5.578	9.500	9.844	11.722	10.278	12.856	13.289

Table 3: The authors report that TOTO significantly outperforms other zero-shot models and baselines across all metrics. MASE and CRPS are normalized by the Seasonal Naive forecast and aggregated using the shifted geometric mean. Rank represents the mean CRPS rank across tasks. For model families with multiple sizes (Moirai, Chronos), the best-performing variant is shown. Similar performance trends are observed on the BOOMLET benchmark.

Metric	Zero Shot					Full Shot				Baselines		
	TOTO	Moirai _{Large}	TimesFM _{2.0}	Chronos _{Bolt-Base}	TabPFN-TS	TEMPO	TTM-R2	PatchTST	TFT	Auto-ARIMA	Auto-ETS	Auto-Theta
MASE ↓	0.673	0.785	0.680	0.725	0.748	0.773	<u>0.679</u>	0.762	0.822	0.964	1.088	0.978
CRPS ↓	<u>0.437</u>	0.506	0.465	0.485	0.480	0.434	0.492	0.496	0.511	0.770	6.327	1.051
Rank ↓	5.495	10.330	8.412	<u>8.309</u>	8.402	8.897	10.103	10.268	11.629	21.608	25.134	24.134

Table 4: On the GIFT-Eval leaderboard, the authors report that TOTO ranks first overall—best in both average Rank and MASE, and second-best in CRPS. MASE and CRPS are normalized against the Seasonal-Naive forecast and combined with a geometric mean; Rank is the average CRPS position. For Moirai and Chronos, only their top-performing variant size is shown.

Dataset	Metric	Toto	Moirai _{Small}	Moirai _{Base}	Moirai _{Large}	Time-MoE _{Base}	Time-MoE _{Large}	Time-MoE _{Ultra}	VisionTS
ETTh1	MAE ↓	0.413	0.424	0.438	0.469	0.424	0.419	0.426	<u>0.414</u>
	MSE ↓	0.435	0.400	0.434	0.510	0.400	<u>0.394</u>	0.412	0.390
ETTTh2	MAE ↓	0.363	0.379	0.382	0.376	0.404	0.415	0.399	<u>0.375</u>
	MSE ↓	<u>0.340</u>	0.341	0.345	0.354	0.366	0.405	0.371	0.333
ETTh1	MAE ↓	<u>0.378</u>	0.409	0.388	0.389	0.415	0.405	0.391	0.372
	MSE ↓	0.396	0.448	0.381	0.390	0.394	0.376	0.356	<u>0.374</u>
ETTm2	MAE ↓	0.303	0.341	0.321	<u>0.320</u>	0.365	0.361	0.344	0.321
	MSE ↓	0.267	0.300	<u>0.272</u>	0.276	0.317	0.316	0.288	0.282
Electricity	MAE ↓	0.242[†]	0.320	0.274	<u>0.273</u>	-	-	-	0.294
	MSE ↓	0.158[†]	0.233	<u>0.188</u>	<u>0.188</u>	-	-	-	0.207
Weather	MAE ↓	0.245	0.267	<u>0.261</u>	0.275	0.297	0.300	0.288	0.292
	MSE ↓	0.224	0.242	<u>0.238</u>	0.259	0.265	0.270	0.256	0.269
Mean	MAE ↓	0.324	0.357	<u>0.344</u>	0.350	-	-	-	0.345
	MSE ↓	0.303	0.327	<u>0.310</u>	0.330	-	-	-	<u>0.309</u>
Best Count		8	0	0	0	0	0	1	3

Table 5: Zero-shot comparison of models on the LSF benchmark. The “Best Count” row shows how often each model achieves the top result for a dataset-metric pair. Dash indicates a model was pretrained on that dataset

Discussion of Results:

- In the BOOM and BOOMLET benchmarks, TOTO significantly outperforms other zero-shot models and baselines across all metrics.
- On the GIFT-Eval leaderboard, the authors report that TOTO ranks first overall—best in both average Rank and MASE, and second-best in CRPS.
- In the LSF benchmark, TOTO consistently outperforms other zero-shot baselines, ranking best on 8 of 12 metrics and achieving the lowest average MAE and MSE. It excels especially on ETTm2, Electricity, and

Weather datasets.

- Gradient Boosted Tree models are missing, but GIFT-Eval contains these models—and since TOTO ranks first, it outperforms these models.
- Although TOTO topped GIFT-Eval at publication, it has since been surpassed by TiRex, a new zero-shot model based on the xLSTM architecture—showing that competition in the zero-shot forecasting field is heating up!

Closing Remarks

Toto is a remarkable model and a huge milestone for the time-series foundation community.

The new approach that TOTO follows by focusing on sparse, intermittent data is very useful and addresses a tough area of time series. The release of the observability BOOM dataset is equally useful.

That said, TOTO isn't without limitations. The model doesn't yet support static variables, and the fine-tuning code has not been released yet. Also, future variables haven't been considered, but TOTO's API has an unused function signature for them—so we may see them in a future version.

Like most time-series foundation models, I believe we may see a new version of TOTO in the future, so stay tuned!

Thank you for reading!

Horizon AI Forecast is a reader-supported newsletter, featuring occasional bonus posts for supporters. Your support through a paid subscription would be greatly appreciated.

Upgrade to paid

[1] Cohen et al. [Toto and BOOM unleashed: Datadog releases a state-of-the-art open-weights time series foundation model and an observability benchmark](#)

[2] Cohen et al. [This Time is Different: An Observability Perspective on Time Series Foundation Models](#)

[3] Cohen et al. [Toto: Time Series Optimized Transformer for Observability](#)

Thank you for reading [AI Horizon Forecast](#) by [Nikos Kafritsas](#)! If you liked this article, feel free to share it!

Share this article!



© 2025 Nikos Kafritsas
[Unsubscribe](#)

Get the app

 Start writing